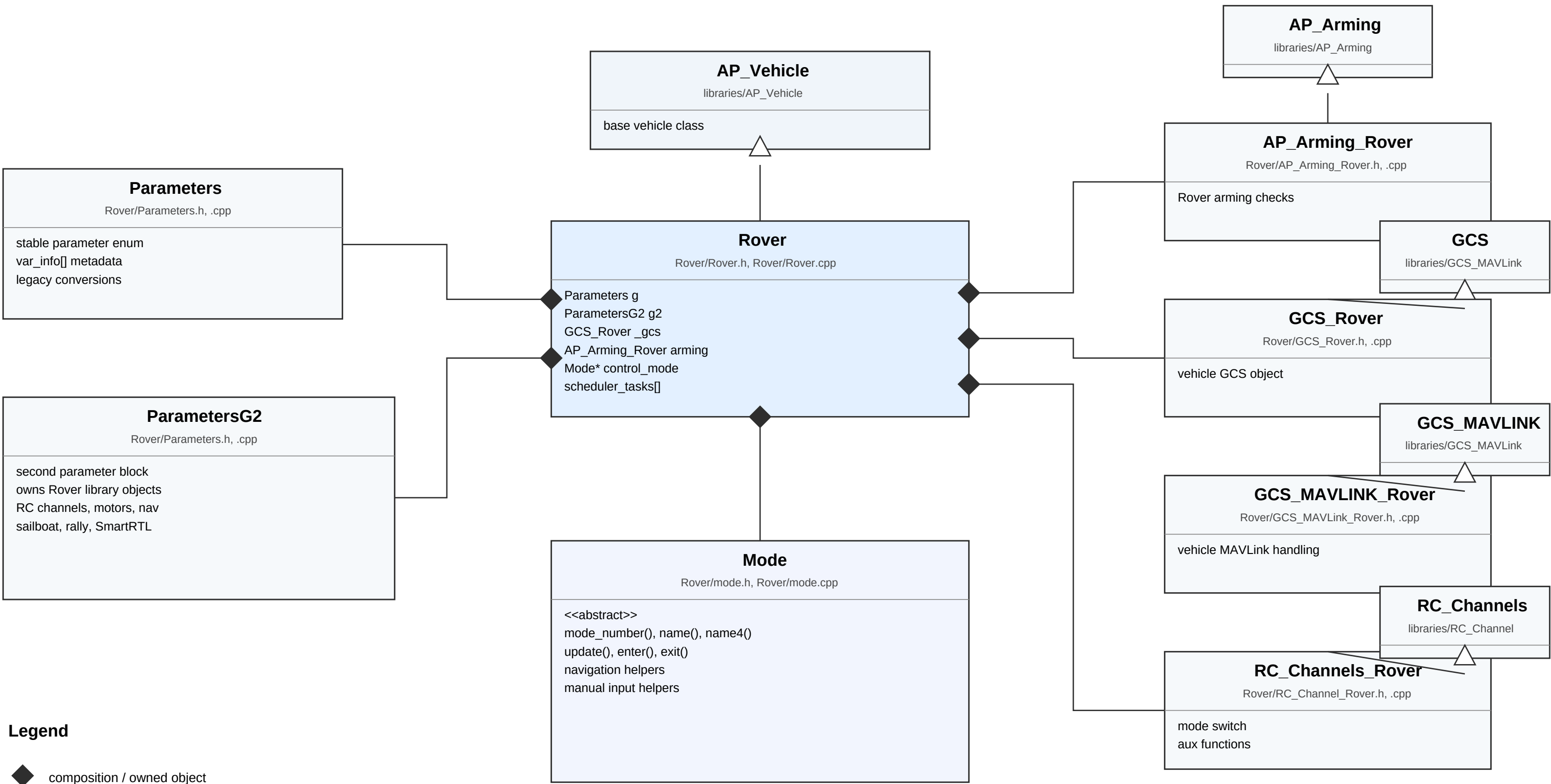


ArduPilot Rover UML - Main Vehicle Structure

Classes are annotated with their local source filenames



Legend

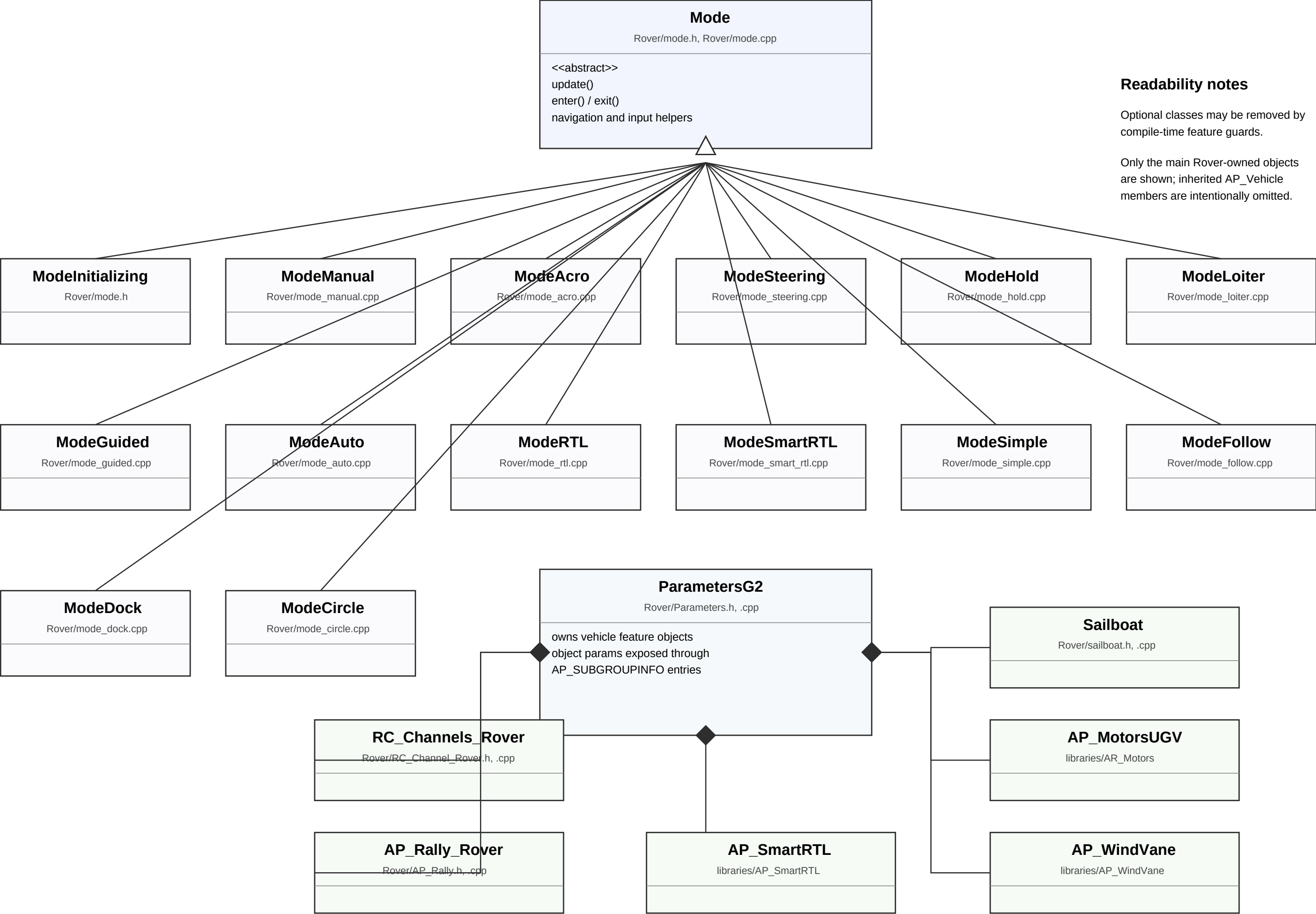
◆ composition / owned object

△ inheritance

This page shows the main class-level shape.
The next page expands modes and ParametersG2 owned objects.

ArduPilot Rover UML - Modes and Owned Objects

Expanded view of Mode subclasses and major ParametersG2 composition



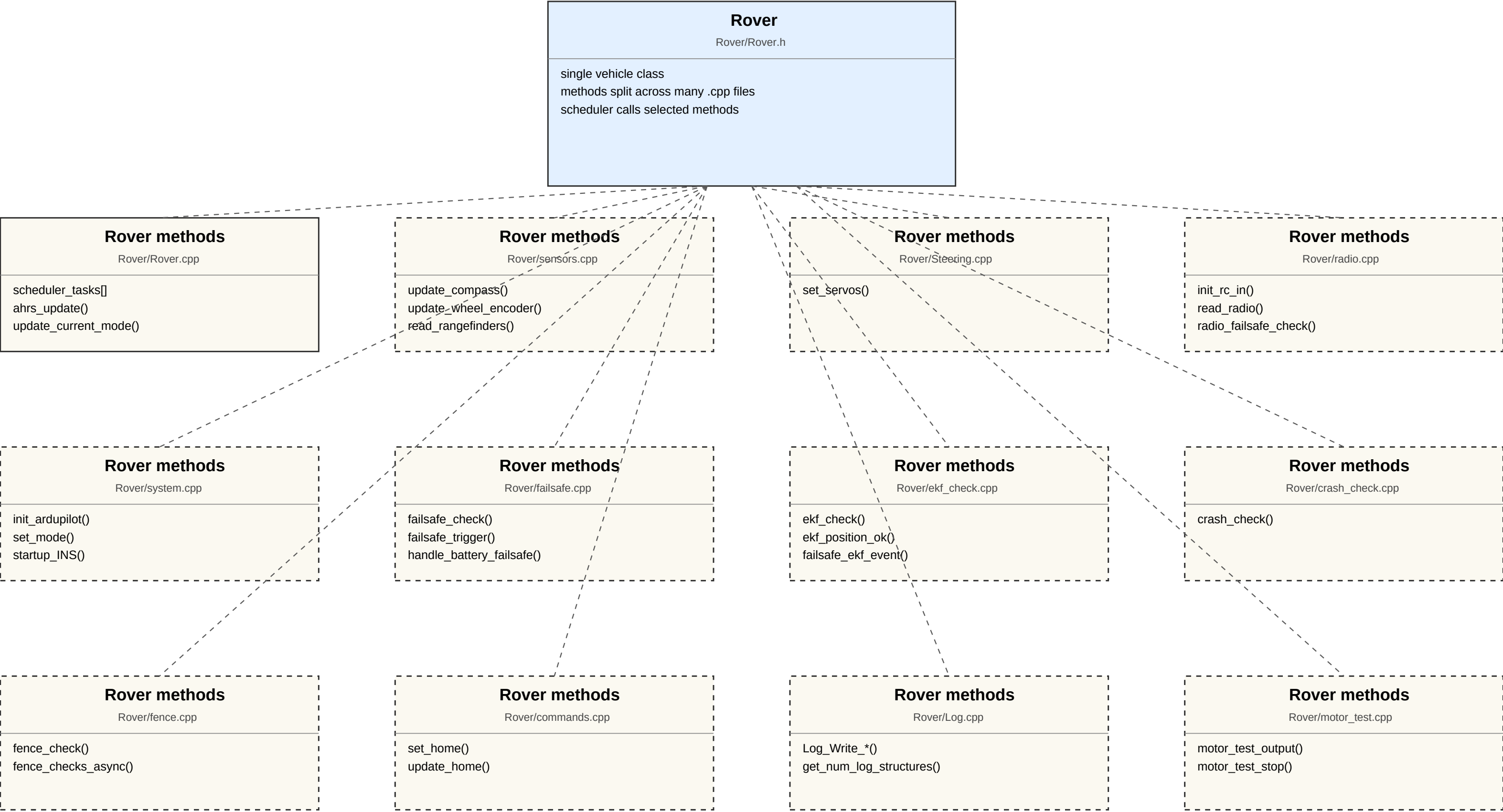
Readability notes

Optional classes may be removed by compile-time feature guards.

Only the main Rover-owned objects are shown; inherited AP_Vehicle members are intentionally omitted.

ArduPilot Rover UML - Rover Method Files

One Rover class is declared in Rover.h; many .cpp files implement its member functions



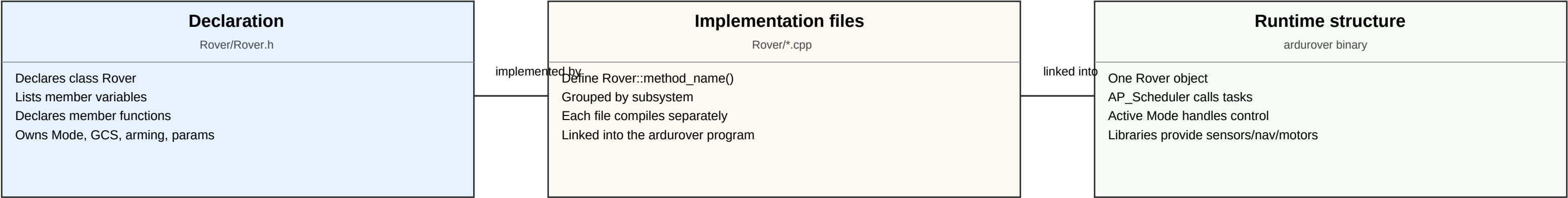
Why sensors.cpp was not on the earlier diagram:

It is not a class. It is one compilation unit that contributes member functions to the Rover class.

The functions are scheduled from Rover.cpp: compass at 10 Hz, wheel encoder at 50 Hz, and rangefinders when enabled.

ArduPilot Rover UML - Reading the Compilation Units

How the class diagram relates to the actual file layout



Important distinction

A C++ class can be declared once and implemented in many compilation units.

That is what Rover does. The scattered files are not separate Rover classes; they are grouped method bodies.

For example, sensors.cpp defines Rover::update_compass(), Rover::update_wheel_encoder(), and Rover::read_rangefinders().

Why ArduPilot uses this layout

- Rover is a large vehicle class with many responsibilities: initialization, radio input, sensors, navigation, output, failsafes, logging, and commands.
- Splitting method implementations by subsystem keeps each file reviewable without changing the object model.
- The scheduler table in Rover.cpp is the best high-level map of which methods run periodically.
- Mode classes are separate classes because each mode has its own behavior and overrides the shared Mode interface.
- Feature classes such as Sailboat, AP_Arming_Rover, and GCS_Rover are separate objects owned by Rover or ParametersG2.

Practical reading order

1. Start at Rover/Rover.h to see owned state and declared methods.
2. Read Rover/Rover.cpp to understand scheduler timing and the main loop.
3. Follow the relevant subsystem file, such as sensors.cpp, radio.cpp, Steering.cpp, or failsafe.cpp.
4. For behavior differences by driving mode, read mode.h and the matching mode_*.cpp file.
5. For parameters and owned library objects, read Parameters.h and Parameters.cpp.